Lecture 07 — The for loop — repetition you can count

Algorithmic Thinking · Week 4, Session 7 — Tuesday 13 October 2026, 17:35–18:55 (Quick exam 3 in the first 20 minutes)

Leopoldo Rodríguez Salazar · Tecnológico de Monterrey

Contents

Overview 1
Opener — Play it a hundred times (10 min) 1
Notes 2
Stretch — if the session was easy 5
After the session 5

Objective. Write a loop over a range you know in advance, say how many times it runs before running it, and use it with an accumulator and a decision.

Overview

Quick exam 3 takes the first twenty minutes. Then Python gives you a word for something you have been writing by hand for a week.

Every count-controlled `while` you wrote — `i = 1`, `while i <= n`, `i = i + 1` — had the same three lines in the same three places. **`for i in range(1, n + 1):`** is those three lines in one. Nothing you can do with `for` is impossible with `while`; `for` is `while` with the start, test and step written where you cannot forget them. That is why half of last week's bugs cannot happen in a `for` loop — and why the other half can.

One idea: **`for` hands you the next value from a sequence until the sequence runs out.** This week the sequence is `range`. In week 7 it will be a list. The loop does not change.

Opener — Play it a hundred times (10 min)

Algorithms to Live By, chapter 9, *Randomness*. Los Alamos, 1946. Stanisław Ulam is recovering from an illness, playing solitaire, and wants to know the probability that a game laid out at random can be won. He tries to compute it — combinatorics — and gives up. Then the thought that named a method: *why not just lay out a hundred games and count how many are winnable?*

That is **Monte Carlo**: when the exact answer is too hard, **repeat a random experiment many times and count**. Ulam and von Neumann used it on the first computer for the bomb's neutron calculations; it is now how banks price risk, how physicists simulate anything, how your phone's weather app says 40 %.

The simplest Monte Carlo you can draw on a napkin: a square 2 by 2, a circle of radius 1 inside it. Throw darts at random. The fraction that land in the circle is the circle's area over the square's: π / 4. So **π ≈ 4 × (darts inside) / (darts thrown)**. No geometry needed beyond "is this point inside the circle?" — which is `x² + y² ≤ 1`.

Your turn, 3 minutes, on paper: write the dart algorithm. `INPUT:` how many darts. `OUTPUT:` an estimate of π. Numbered steps. You will need a counter for the darts inside, a loop that runs *exactly* N times, and one decision per dart. **Underline the line that says how many times the loop runs.** Then the pivot: *that underlined line is today's keyword.*

Notes

The same loop, twice

Open `01_while_vs_for.py`:

```
i = 1
while i <= 5:
    print(i)
    i = i + 1

for i in range(1, 6):
    print(i)
```

Run it: `1 2 3 4 5`, twice. Point at the three parts of the `while` — start, test, step — and find them in the `for` line: `range(1, 6)` *is* the start (1), the test (below 6) and the step (+1). The `for` line cannot forget its step. It cannot put the step outside the block. It cannot run forever. Three of last week's bugs, gone — and one new one: **6 is not included.**

`range`, three shapes

Open `02_range.py`. Predict each before running:

	gives	rule
`range(5)`	`0 1 2 3 4`	one number: from 0, up to but not including
`range(2, 6)`	`2 3 4 5`	two: from, up to but not including
`range(10, 0, -2)`	`10 8 6 4 2`	three: from, up to, step (can be negative)
`range(3, 3)`	nothing	zero turns, no error

The rule to say out loud until it is boring: **the end is not included.** `range(1, 101)` runs 100 times. `range(1, n + 1)` is "from 1 to n". If you want "to n" you write `n + 1` — every time, forever. This is `for`'s one off-by-one, and it replaces all of `while`'s.

The accumulator, again

Open `03_sum_and_factorial.py`:

```
def sum_to(n):
    total = 0
```

```
    for i in range(1, n + 1):
        total = total + i
    return total

def factorial(n):
    product = 1
    for i in range(1, n + 1):
        product = product * i
    return product
```

`sum_to(10)` → `55` — the same program as lecture 5's `05_sum_to_n.py`, two lines shorter and with the 55-becomes-65 bug now impossible (there is no step line to misplace). `factorial(5)` → `120`. Now the shell question: **`factorial(0)`**. The loop runs zero times (`range(1, 1)` is empty) and the function returns `product` untouched — `1`. Which is right: 0! = 1. The start value of an accumulator is *the answer when there is nothing to accumulate*: `0` for a sum, `1` for a product, `0` for a count. Had we started `product = 0`, every factorial would be 0 and no error would ever tell us.

A decision inside the repetition

Open `04_for_with_if.py`:

```
def count_multiples(n, limit):
    count = 0
    for i in range(1, limit + 1):
        if i % n == 0:
            count = count + 1
    return count
```

`count_multiples(7, 100)` → `14`. The `if` lives four spaces inside the `for`; the accumulator grows only on the turns where the decision says yes. **Count, keep, or skip** — most loops you will ever write are one of these three. `sum_of_divisors(12)` → `16` (1 + 2 + 3 + 4 + 6) is *keep*: Friday's lab asks for the same divisors, printed; Friday's session asks what number equals the sum of its own divisors.

Monte Carlo, in eleven lines

Open `05_monte_carlo_pi.py` — the opener:

```
import random

def estimate_pi(trials):
    inside = 0
    for i in range(trials):
        x = random.random()
        y = random.random()
        if x * x + y * y <= 1:
            inside = inside + 1
    return 4 * inside / trials
```

Run it with `100`, then `100000`, then `100000` again. Three different answers, the big ones near 3.14. Two things to notice. The loop variable `i` is never used inside the block —

that is allowed and common: *do this `trials` times* is all we wanted from it. And `random.random()` gives a decimal from 0.0 to 1.0, so we throw at one quarter of the square; the ratio is the same.

A flag

Open `06_is_prime.py`:

```
def is_prime(n):
    if n < 2:
        return False
    found_divisor = False
    for d in range(2, n):
        if n % d == 0:
            found_divisor = True
    return not found_divisor
```

`is_prime(97)` → `True`; `is_prime(9)` → `False`; `is_prime(1)` → `False` (the first `if`). `found_divisor` is a **flag**: it starts `False`, it becomes `True` the moment the loop finds a reason to say no, and it never goes back — later turns can only set it to `True` again. After the loop, the flag *is* the answer, negated.

Why not stop the loop as soon as a divisor is found? Python has a word for that (`break`), and we will not use it in this course: a loop you can leave from the middle is a loop whose exit you can no longer read on the `for` line. The flag costs a few wasted turns and keeps the termination obvious. (If you have seen `break`, you may use it in the stretch problems; not in exams.)

Failure-first: three `for` loops that run and are wrong

Open `07_three_bugs.py`. Predict, then run.

1. **`range(1, n)` for "1 to n"** — prints 1 to 4 for `n = 5`. The '`+ 1is missing. This is the onefor`' bug, and it is on every exam from now on.
2. **The accumulator reset inside the loop** — `total = 0` *inside* the block: every turn starts from nothing, and the sum of 1, 2, 3 comes out as 3. The start line belongs *before* the loop; indentation decides.
3. **Changing `i` inside the `for`** — `i = i + 10` does nothing to the next turn; `for` hands out the next value of the range regardless. If you need to steer the counter, you wanted a `while`.

In-class activity 3 — Primes, counted *(individual, 20 min, graded — the third of the four in-class activities, 4 % of the course)*

Two functions and a `main`. One file.

1. **`is_prime(n)`** — returns `True` when `n` is prime, `False` otherwise (`n < 2` is never prime). A `for` over `range(2, n)` and a **flag**. No `break`.
2. **`count_primes(limit)`** — returns how many primes there are from 2 to `limit` *inclusive*. A `for` that calls `is_prime` — a loop around a loop. Say in your algorithm why `limit + 1` appears.
3. **`main()`** — reads `limit`; if it is below 2, displays `Nothing to count.`; otherwise displays the count. Last line calls `main()`.

Algorithm first, on paper — `INPUT:`, `OUTPUT:`, numbered steps, the `FOR` lines written with their ranges, the flag named. Then the program in exam format; then the shell tests, under the algorithm:

```
is_prime(1)        False
is_prime(2)        True
is_prime(9)        False
is_prime(97)       True
count_primes(10)   4        (2, 3, 5, 7)
count_primes(2)    1
count_primes(1)    0
count_primes(100)  25
```

Submit: photo of the algorithm + `A_Matricula_ic3.py` → Canvas → *In-class activity 3: for loops — primes, counted* — by **Sunday 18 October, 23:59**, but the work happens now.

> Rubric: algorithm with input/output, both `FOR` ranges stated and the flag named 40 % · program passes all eight shell tests 40 % · `count_primes(2)` → 1 and `count_primes(1)` → 0 handled on purpose (the `+ 1`, and the `< 2` rule) 20 %.

Finished early? `is_prime` tests every `d` up to `n - 1`. Argue, in one sentence under the algorithm, why testing up to `n // 2` is enough — and then why the square root of `n` is enough.

Stretch — if the session was easy

Optional, ungraded, harder than they look.

1. **How many darts?** Run `estimate_pi(n)` ten times each for `n` = 100, 10 000 and 1 000 000 and write down the spread (largest minus smallest). Then state the rule: when `n` grows by 100×, the spread shrinks by about how much? (It is a famous and disappointing number.)
2. **Perfect.** A number is *perfect* when it equals the sum of its proper divisors: 6 = 1 + 2 +
3. Using `sum_of_divisors`, find every perfect number below 10 000 with a loop around it. There are four. How long did your program take, and what single change to `sum_of_divisors` (which `range` end?) halves that time without changing an answer?
3. **The `for` that must be a `while`.** Rewrite `play(secret)` from lecture 6 as a `for` loop with at most 7 turns — `for attempt in range(7):` — that still stops early when the guess is right, *without* `break`. You will need a flag in the test of… wait, a `for` has no test. Explain, in two sentences, what you had to do, and why the `while` version is better.

After the session

Homework 5 — `assignments/hw-05-for.md`. Due Sunday 18 October, 23:59, on Canvas. Two functions and a `main`: `is_perfect(n)` with a `for`, and `longest_collatz(limit)` — a `for` around your own `collatz_steps` from Homework 4. Algorithm on paper, shell tests written down, one sentence per `for` saying how many times it runs.

Also before Friday:

1. **Lab 4 — Friday 16:30**, with Luis: the academia's *for y while* lab — `factores_de`, `pares`, a menu and a `main` that repeats until *Salir*, in one file `D_matricula.py`. Same lab as every group. Bring the laptop.
2. **Friday's session: `for` in practice — and the Situación problema is formally released.** Reread the brief on the site (`situacion-problema.html`); think about who your partner is.
3. **Stretch for credit** — if you handed in both week-3 stretch exercises by Sunday, the grades go up this week. One use per student; the rule is on Canvas.
4. **Quick exam 4 is Tuesday 20 October**, first 20 minutes: the `for` loop. Then the **midterm on Friday 23 October** — everything from weeks 1 to 4.